به نام خدا

سیستم‌عامل - بهار ۱۴۰۵

پروژه اول : فراخوانی‌های سیستمی

دستیار آموزشی این مجموعه: سجاد حسن‌پور / سپنتا غنودی

مهلت تحویل: ۴ اردیبهشت ۱۴۰۵ (صفحه درس)

مقدمه و اهداف پروژه

در برنامه‌نویسی سطح سیستم، شناخت تئوری مفاهیم کافی نیست؛ بلکه درک واقعی زمانی حاصل می‌شود که برنامه شما مستقیماً با هسته سیستم‌عامل [۱] وارد تعامل شود. هدف اصلی این پروژه، درگیری مستقیم شما با فراخوانی‌های سیستمی [۲] در سیستم‌عامل لینوکس است. در این مسیر، با چالش‌های واقعی برنامه‌نویسی سیستمی روبرو شده و مفاهیم بنیادینی نظیر مدیریت پردازش‌ها [۳] ، مدیریت سیگنال‌ها [۴] و ارتباط بین‌پردازشی [۵] از طریق صف‌های پیام [۶] را به صورت کاملاً عملی و کدنویسی‌شده لمس خواهید کرد.

در این پروژه، شما یک چت‌روم ترمینالی همزمان [۷] و امن را طراحی و پیاده‌سازی خواهید کرد. در این محیط، دو کاربر می‌توانند در یک سیستم یکسان به تبادل پیام‌های رمزنگاری‌شده بپردازند و قابلیتی جذاب برای ارسال پیام‌های زمان‌دار و خودتخریب‌گر [۸] در اختیار داشته باشند.

[1] Kernel
[2] System Calls
[3] Process Management
[4] Signals
[5] Interprocess Communication
[6] Message Queues
[7] Full-Duplex
[8] Self-Destructing

فاز اول: بستر ارتباطی اولیه

در این بخش شما بستر ارتباطی اولیه را ایجاد می‌کنید تا دو کاربر بتوانند یکدیگر را پیدا کرده و به یک صف پیام مشترک متصل شوند. برای این هماهنگی از یک فایل موقت پنهان (مانند `.ghost_session`) استفاده می‌شود.

صف پیام یکی از ابزارهای ارتباط بین‌پردازه‌ای در سیستم‌عامل است که دقیقاً مانند یک صندوق پستی عمل می‌کند؛ یک برنامه پیام را در آن قرار می‌دهد و برنامه دیگر آن را می‌خواند. برای اینکه دو برنامه کاملاً مجزا بتوانند از یک صندوق مشترک استفاده کنند، برنامه دوم باید بداند صندوقی که برنامه اول ساخته چه شناسه‌ای (ID) دارد. ما این شناسه را از طریق همان فایل پنهان به دست کاربر دوم می‌رسانیم.

برای پیاده‌سازی، برنامه‌ای که زودتر اجرا می‌شود (کاربر اول) باید صف پیام را بسازد و برنامه‌ای که دیرتر می‌آید (کاربر دوم) باید به آن متصل شود. برای تشخیص اینکه کدام کاربر هستید، می‌توانید از تابع `open` با پرچم‌های[9] `O_CREAT | O_EXCL` استفاده کنید.

اگر فایل با موفقیت ساخته شد، یعنی شما کاربر اول هستید؛ پس باید با استفاده از تابع `msgget` یک صف پیام ایجاد کرده و شناسه آن را با فراخوانی سیستمی `write` داخل این فایل بنویسید. اما اگر ساخت فایل خطا داد (یعنی فایل از قبل توسط نفر اول ساخته شده است)، شما کاربر دوم هستید و فقط باید فایل را باز کرده و شناسه صف پیام را از روی آن بخوانید تا به همان صف متصل شوید.

نکته مفید برای دیباگ: برای مشاهده‌ی صف‌های پیامِ ایجاد شده در سیستم، می‌توانید از دستور زیر در ترمینال لینوکس استفاده کنید:

```
ipcs -q
```

این دستور به شما کمک می‌کند تا مطمئن شوید صف پیام شما به درستی در سیستم‌عامل ثبت شده است.

بر اساس ساختار برنامه، وضعیت اجرای شما باید در ترمینال به کاربر نمایش داده شود. در ابتدای اجرای برنامه، باید کل صفحه ترمینال پاک شده و پیام خوش‌آمدگویی زیر چاپ شود:

```
=== Welcome to Ghost Chat ===
```

سپس بسته به اینکه کاربر اول باشید یا دوم، پیام متفاوتی در ترمینال نمایش داده می‌شود. اگر برنامه به عنوان کاربر اول اجرا شود، باید پیام انتظار برای ورود نفر دوم را چاپ کند:

```
Waiting for User 2 to join...
```

و اگر برنامه به عنوان کاربر دوم اجرا شود و با موفقیت شناسه را خوانده و متصل شود، باید این پیام را نمایش دهد:

```
Connected to User 1's chat!
```

فاز دوم: چت همزمان (Full-Duplex)

در این بخش شما قابلیت چت همزمان را پیاده‌سازی می‌کنید تا هر دو کاربر بتوانند بدون منتظر ماندن برای یکدیگر، همزمان پیام بفرستند و دریافت کنند.

در یک برنامه خط‌فرمانی عادی، وقتی سیستم منتظر ورودی شماست (مثلاً با فراخوانی `read` از کیبورد)، برنامه متوقف می‌شود و تا زمانی که چیزی تایپ نکنید، نمی‌تواند پیام‌های جدیدی که از طرف مقابل به صف پیام آمده است را دریافت و چاپ کند. برای حل این مشکل، باید برنامه را به دو جریان کاری همزمان تقسیم کنیم.

[9] flag

برای پیاده‌سازی این بخش، از فراخوانی سیستمی fork استفاده می‌کنیم. با فراخوانی fork، برنامه شما به دو پردازه «پدر» و «فرزند» تقسیم می‌شود که به صورت همزمان اجرا می‌شوند:

۱. **پردازه ارسال‌کننده (پدر):** در یک حلقه بی‌نهایت، پیام‌ها را از ورودی استاندارد (کیبورد) می‌خواند و با استفاده از فراخوانی سیستمی msgsnd آن‌ها را داخل صف پیام قرار می‌دهد.

۲. **پردازه دریافت‌کننده (فرزند):** در یک حلقه بی‌نهایت دیگر، با استفاده از فراخوانی سیستمی msgrcv منتظر رسیدن پیام‌های جدید در صف می‌ماند و به محض دریافت، آن‌ها را روی صفحه ترمینال چاپ می‌کند.

از آنجا که هر دو کاربر از یک صندوق (صف پیام) مشترک استفاده می‌کنند، اگر دقت نکنید ممکن است برنامه شما پیامی که خودتان فرستاده‌اید را به جای پیام طرف مقابل از صف بخواند!

خوشبختانه ساختار پیام‌ها در Message Queue دارای فیلدی به نام mtype (نوع پیام) است. برای تفکیک پیام‌ها، باید توافق کنید که کاربر اول پیام‌هایش را با نوع ۱ بفرستد و پردازه دریافت‌کننده‌اش فقط منتظر پیام‌های نوع ۲ باشد. در طرف مقابل، کاربر دوم باید پیام‌هایش را با نوع ۲ بفرستد و فقط پیام‌های نوع ۱ را از صف بخواند.

خروجی ترمینال در این فاز باید به گونه‌ای باشد که پیام‌های ارسالی شما و پیام‌های دریافتی از شخص مقابل به وضوح تفکیک شوند. پیشنهاد می‌شود فرمت نمایش به شکل زیر باشد:

پیام‌هایی که خودتان می‌فرستید:

```
You: Hello, are you there?
```

پیام‌هایی که از طرف مقابل دریافت می‌کنید:

```
Stranger: Yes, I'm here!
```

فاز سوم: پیام‌های خود تخریب‌گر

در این بخش شما پیام های خود تخریب‌گر را ایجاد می‌کنید.این پیام ها به این صورت عمل میکنند که پس از مدت زمان مشخصی (که برای هر پیام مشخص می‌شود)از تاریخچه پیام‌های هر دو کاربر حذف خواهند شد. این پیام‌ها به صورت زیر ارسال می‌شوند.

```
BURN <seconds> <message>
```

جهت پیاده‌سازی این قابلیت، پس از تبادل هر پیام، با فراخوانی تابع fork یک پردازه جدید ایجاد نمایید تا از مسدود شدن[10] برنامه اصلی جلوگیری شود. در پردازه فرزند، پس از سپری شدن زمان تعیین‌شده، پیام مربوطه باید از تاریخچه چت هر دو کاربر حذف گردد. برای این کار، پیام‌های معتبر را به یک فایل موقت منتقل و سپس آن را جایگزین فایل اصلی کنید. فرض بر این است که در طول فرآیند حذف، پیام جدیدی در فایل اصلی ثبت نخواهد شد.

جهت پاک‌سازی پیام از ترمینال، ابتدا صفحه را خالی و سپس بر اساس فایل تاریخچه جدید، بازنویسی نمایید. عملیات حذف باید دوطرفه باشد؛ به این معنا که هم فرستنده و هم گیرنده موظف هستند پس از شناسایی پیام‌های منقضی‌شده و اتمام مهلت مقرر، آن‌ها را از فایل تاریخچه و محیط ترمینال خود حذف کنند.

برای خالی کردن ترمینال می‌توانید از قطعه کد زیر استفاده کنید:

```
write(1, "\033[H\033[J", 6);
```

این قطعه کد نشانگر ترمینال را به ابتدای صفحه برده(گوشه بالا، سمت چپ) و هر چه پس از آن باشد را پاک می‌کند.

[10] Blocking

فاز چهارم: خروج امن

در این فاز، قابلیت خروج از برنامه پیاده‌سازی می‌شود. با ارسال عبارت EXIT توسط هر یک از کاربران، برنامه باید خاتمه یافته و فایل‌های تاریخچه چت و ghost_session از حافظه حذف شوند تا در اجرای بعدی، برنامه با تاریخچه‌ای تهی شروع به کار کند. جهت پاک‌سازی فایل‌ها می‌توانید از فراخوانی سیستمی unlink استفاده نمایید. همچنین لازم است صف پیام ایجاد شده در برنامه نیز حذف گردد (برای این منظور می‌توانید از فراخوانی سیستمی msgctl استفاده کنید). در صورتی که یکی از کاربران عبارت EXIT را ارسال کند، باید در صفحه کاربر دیگر پیام زیر نمایش داده شود:»

```
[SYSTEM] The other user has left the chat
```

علاوه بر این، پس از ارسال دستور خروج توسط یکی از کاربران، برنامه‌ی او باید با فرستادن یک پیام کنترلی، دستور خاتمه را به برنامه‌ی کاربر مقابل نیز منتقل کند تا هر دو برنامه به‌طور کامل بسته شوند. با توجه به اینکه برنامه‌ی هر کاربر شامل دو پردازه (یک پردازه جهت دریافت و یک پردازه جهت ارسال پیام) است، هر دو پردازه باید خاتمه یابند. برای انجام این عملیات می‌توانید از فراخوانی سیستمی kill استفاده کنید. همچنین دقت داشته باشید که پردازه والد باید تا اتمام کامل تمامی پردازه‌های فرزند صبر کرده (wait) و سپس خود خاتمه یابد

نکات تحویل

- کدهای شما می‌بایست تنها به زبان C نوشته شده و در سیستم‌عامل Linux کامپایل و اجرا شوند. در غیر این صورت، نمره‌ای به پروژه تعلق نخواهد گرفت.
- محدودیت‌های پیاده‌سازی: هسته‌ی اصلی این پروژه بر پایه‌ی فراخوانی‌های سیستمی است.
  - برای عملیات کار با فایل‌ها و ورودی/خروجی، **اکیداً** از فراخوانی‌های سیستمیopen، read، write و close استفاده کنید.
  - استفاده از توابع کتابخانه‌ای استاندارد مانند fopen، fprintf، scanf و موارد مشابه ممنوع است.
  - استفاده از توابع کمکی رشته‌ای مانند sprintf، strlen و atoi بلامانع است.
- تمامی فایل‌های خود از قبیل سورس‌کدها را در پوشه‌ای تحت عنوان OS-CA1-StudentNumber (که در آن StudentNumber شماره دانشجویی شماست) قرار داده، آن را zip کنید و در سامانه بارگذاری نمایید.
- توجه داشته باشید که در هنگام تحویل، تنها فایل‌های بارگذاری‌شده در سامانه پذیرفته خواهند شد و ملاک نمره‌دهی خواهند بود.

موفق باشید!